



End-to-End ML System — Project Documentation

Viasat History CEE | XGBoost · FastAPI · Streamlit | 02 May 2026

1. Project Overview

This project builds an intelligent TV promo scheduling system for Viasat History CEE. It uses seven years of broadcast scheduling data to train two machine learning models: a **classifier** that predicts whether a promo should be repositioned in its break, and a **regressor** that estimates how much effectiveness uplift that move would deliver. The models are served through a production-ready **FastAPI** REST service and visualised in a **Streamlit** dashboard that replicates the Ancast Nowcasting operator interface.

■ *Advisory output only. Recommendations are surfaced to operators who decide whether to act. No schedule changes are made automatically.*

Layer	File	Purpose
Training pipeline	ctv_promo_pipeline.py	Loads data, trains models, saves artifacts
ML artifacts	model/	promo_move_classifier.pkl, promo_uplift_regressor.pkl, inference_sche
REST API	main.py	FastAPI service — /predict, /predict/batch, /reload, /health
Frontend	streamlit_app.py	Ancast Nowcasting operator UI
Schema recovery	generate_schema.py	Extracts inference_schema.json from pkl files when missing
Tests	tests/test_api.py	20 pytest unit tests with mocked models
Container	Dockerfile / docker-compose.yml	Containerised deployment

2. Problem Statement

TV promos compete for limited break slots. A promo placed at the wrong position in a break (e.g., third of four when first would perform better) loses audience impact. Schedulers currently make these decisions manually, often without access to real-time signals like weather, streaming competition, or audience fatigue.

Two prediction tasks

- **Classification** — should_move: should this promo be repositioned? (0 = keep, 1 = move)
- **Regression** — uplift_if_optimised: if moved, how much effectiveness gain is expected?

Why this is hard

- Class imbalance: roughly 80% of promos should be kept, only 20% should move.
- Data leakage risk: post-broadcast outcome columns (actual position, was_missed) must never be used as features.
- Temporal drift: 7 years of data spans COVID disruption and a streaming revolution — a random train/test split would cheat.
- Categorical cardinality: promo_title and neighbour titles have hundreds of unique values with no natural ordering.

3. Dataset

The training dataset covers 7 years of Viasat History CEE broadcast scheduling. Each row represents one promo event at a specific time in a specific break.

Feature Group	Feature Names	Type
Identity / title	channel, promo_title, content_type, event_type, genre_guess	Categorical
Scheduling	hour, minute, day_of_week, is_weekend, time_band	Numeric / Cat
Break placement	promo_position_type, promo_in_break, break_event_position, break_total_events, break_position_pct, preceded_by_break, lead_to_next_program_min	Numeric / Cat
Neighbour context	prev_promo_title, next_promo_title	Categorical
Weather	weather_station, weather_tmax_c, weather_tmin_c, weather_rain_mm, weather_sun_hours, weather_bad_index, weather_indoor_viewing_index	Numeric / Cat
OTT / streaming	ott_avg_watch_pct, ott_dropoff_prob, ott_hook_strength, ott_visual_intensity	Numeric
Competitive	netflix_popularity_score, netflix_rating_mean	Numeric
Synthetic / derived	synthetic_premiere_probability, synthetic_production_year_mean	Numeric
Promo health	promo_fatigue_index, attention_context_score	Numeric

Leaky columns removed (7): actual_status, actual_start_offset_seconds, actual_duration_seconds, duration_diff_seconds, was_missed, observed_effectiveness_score, best_possible_score. These describe what happened after the promo aired and are not available at scheduling time.

4. ML Pipeline (ctv_promo_pipeline.py)

4.1 Eight critical fixes over the original prototype

FIX 1 — Drop leaky features

7 post-hoc outcome columns removed. Using them would give perfect training accuracy but zero real-world performance because they don't exist at scheduling time.

FIX 2 — Time-based train/test split

Data is sorted by datetime and split at the 80th percentile. The model trains on older data and is tested on the most recent 20%. A random split lets the model see future data during training — a form of leakage.

FIX 3 — Keep df_raw for inference

The raw DataFrame (before encoding) is preserved. The original notebook encoded df in place, then re-encoded it at inference time, turning every categorical to UNKNOWN.

FIX 4 — Handle class imbalance

scale_pos_weight = n_negative / n_positive (~4.18) is passed to XGBoost. Evaluation uses PR-AUC (area under the precision-recall curve) not accuracy, because accuracy is meaningless on imbalanced data.

FIX 5 — Regressor trained only on should_move=1 rows

Training the regressor on all rows is misleading — 80% of rows have uplift ≈ 0 (the 'keep' rows), so predicting zero looks impressive but is useless. The regressor only sees rows where a move is actually recommended.

FIX 6 — XGBoost native categorical support

Columns are cast to pandas category dtype and enable_categorical=True is set. LabelEncoder creates false ordinal relationships (title_id=7 is not 'between' 6 and 8).

FIX 7 — Early stopping

n_estimators=1000 with early_stopping_rounds=30. Training stops when the validation PR-AUC stops improving for 30 consecutive rounds, preventing overfitting.

FIX 8 — No fillna(0) on numerics

NaN values are left as-is. XGBoost learns a 'default direction' for each tree split when a feature is missing, which is statistically correct. Filling with 0 would conflate 'missing' with 'zero rain' or 'zero position'.

4.2 Classifier details

The classifier predicts **should_move** (0 or 1). It outputs a probability; the default decision threshold is 0.50, but this can be adjusted for precision/recall trade-offs.

Parameter	Value	Why
n_estimators	1000 (capped by early stopping)	Generous upper bound; early stopping finds the real optimal
max_depth	6	Deep enough for interactions; shallow enough to generalise
learning_rate	0.05	Slow learning = better generalisation than 0.1 or 0.3
subsample	0.9	Stochastic boosting reduces overfitting
colsample_bytree	0.9	Random feature subsets per tree
min_child_weight	3	Prevents splits on tiny leaf groups
scale_pos_weight	~4.18	Compensates for 80/20 class imbalance
eval_metric	aucpr	PR-AUC is the correct metric for imbalanced classification
enable_categorical	True	Native XGBoost categorical handling — no label encoding ne
early_stopping_rounds	30	Stop if PR-AUC doesn't improve for 30 rounds

4.3 Regressor details

The regressor predicts **uplift_if_optimised** — the expected effectiveness gain if this promo is repositioned. It is trained *only* on rows where should_move=1 and is only called at inference time when the

classifier also predicts `should_move=1`. Objective is `reg:squarederror` (MSE minimisation).

4.4 Inference pipeline

At prediction time, the following steps run for every incoming row:

- 1. Build a single-row (or batch) DataFrame from the JSON payload.
- 2. Call `prepare_for_inference()`: select exactly the 37 training features, encode categoricals against the training vocabulary (unseen values become NaN).
- 3. Convert any object-dtype numeric columns to float (`None` → NaN).
- 4. Classifier predicts `move_probability`.
- 5. If `move_probability` ≥ 0.5 → `should_move = 1` → regressor predicts uplift.
- 6. If `move_probability` < 0.5 → `should_move = 0` → `uplift = 0.0` (regressor not called).
- 7. Return `should_move`, `move_probability`, `predicted_uplift`.

5. FastAPI Service (main.py)

The API serves predictions over HTTP. It loads model artifacts once at startup and keeps them in memory for low-latency inference. All endpoints are documented automatically at <http://localhost:8000/docs> (Swagger UI).

5.1 Endpoints

Method	Path	Description
GET	/health	Returns model load status, schema loaded flag, feature count. Returns 503 if models fa
POST	/predict	Single promo prediction. Accepts a JSON object with any subset of the 37 feature fields
POST	/predict/batch	Batch prediction for up to 500 promos in one request. Single DataFrame pass — much
POST	/reload	Hot-reload model artifacts from disk without restarting the server. Use after updating pk

5.2 Request / Response

Request body (/predict):

```
{
  "channel": "Viasat History CEE",
  "promo_title": "Alexander's Lost World - Tue @ 2100",
  "genre_guess": "history_doc",
  "hour": 21,
  "promo_in_break": 1,
  "break_position_pct": 0.5,
  "promo_fatigue_index": 0.85,
  "attention_context_score": 0.30
}
```

Response:

```
{
  "promo_title": "Alexander's Lost World - Tue @ 2100",
  "datetime": null,
  "should_move": 1,
  "move_probability": 0.7812,
  "predicted_uplift": 0.3245
}
```

5.3 Key design decisions

- **All fields optional.** Missing fields become NaN; XGBoost handles them via learned default directions.
- **Batch uses a single model call.** `predict_proba()` is called once on the full DataFrame, not once per row.
- **Regressor gating.** The regressor is never called unless `should_move=1`, matching the training setup exactly.
- **Atomic reload.** `/reload` only updates `_state` after both models load successfully, so a failed reload never breaks live traffic.
- **Structured logging.** Every request logs method, path, status code, and latency in milliseconds.

- **CORS enabled.** Configurable via CORS_ORIGINS environment variable.

5.4 Environment variables

Variable	Default	Description
MODEL_DIR	model	Path to directory containing pkl files and inference_schema.json
CORS_ORIGINS	*	Comma-separated list of allowed CORS origins
BATCH_LIMIT	500	Maximum number of items in a single /predict/batch request

6. Streamlit Frontend (streamlit_app.py)

The Streamlit app implements the Ancast Nowcasting operator interface — a scheduling dashboard that shows today's promo schedule enriched with live ML predictions. It calls /predict/batch on the FastAPI service and falls back to deterministic mock predictions when the API is unreachable.

6.1 UI components

Component	Description
Header bar	Dark navy bar with channel selector, date navigation, and four live stat cards (flags raised, time since last update, etc.).
Advisory banner	Blue info strip reminding operators that all output is advisory — no automatic schedule changes.
Daypart tabs	Breakfast / Daytime / Peak / Late / All day — filters the table to the selected part of the day.
Filter dropdowns	Content type, genre, flag status (Open only / On track / All), sort order.
Search box	Live title search filtering rows as you type.
Schedule table	TX TIME, TITLE (with genre tag), DURATION, TYPE, FLAG badge, ACTION, UPLIFT, RATIONALE columns.
Flag badges	Review — swap (red, probability ≥ 0.70), Review — move (orange, probability 0.50–0.69), On track (green, probability ≥ 0.70).
Footer	API live/offline indicator, count of flags shown, auto-refresh countdown.

6.2 Data flow

1. On load, check /health to determine if the API is reachable.
2. If API live → POST /predict/batch with all 15 schedule items' real feature values.
3. If API offline → compute deterministic mock predictions locally (same formula every time so the UI is stable).
4. Merge predictions back into the schedule rows.
5. Apply user-selected filters (daypart, genre, flag, search).
6. Render the HTML table with coloured badges.
7. Auto-rerun every 30 seconds (st.rerun() + st.cache_data TTL).

7. Supporting Files

7.1 generate_schema.py

When inference_schema.json is missing (e.g., after copying pkl files from Kaggle without the schema), this script recovers it directly from the saved XGBoost model. XGBoost stores feature_names_in_ and feature_types_ on the model object. Run: **python generate_schema.py**. Limitation: category vocabularies cannot be recovered from pkl files alone — the schema will have empty category_values, meaning unseen strings pass through as NaN.

7.2 tests/test_api.py

20 pytest unit tests covering all endpoints. Models are mocked using unittest.mock.MagicMock so the test suite runs without any pkl files. Tests cover:

- Health endpoint — OK response, 503 when not ready, schema-not-loaded flag.
- Predict — minimal payload, response shape, passthrough fields, should_move=0/1 logic, regressor not called when should_move=0, field validation (hour out of range, break_position_pct > 1).
- Predict/batch — basic batch, empty batch rejected, batch limit enforced, per-item passthrough.
- Reload — success path, failure returns 500.

Run: **pytest tests/**

7.3 Dockerfile + docker-compose.yml

The Dockerfile builds a Python 3.11-slim image. The model directory is volume-mounted in docker-compose.yml so pkl files can be updated without rebuilding the image. A built-in Docker healthcheck polls /health every 30 seconds. Two uvicorn workers are configured for better CPU utilisation with XGBoost inference.

```
docker compose up --build
```

8. How Missing Input Values Are Handled

All 37 input fields are optional. When a field is not provided it becomes NaN in the inference DataFrame. XGBoost handles this through a mechanism called **learned default directions**.

How it works

During training, for every split in every tree XGBoost evaluates two choices: when this feature is NaN, send the row left or send it right? It picks whichever direction reduces training loss more. That choice is stored permanently in the model. At inference time, any missing feature follows that stored path through all 100+ trees automatically.

Feature priority tiers

Tier	Fields	Impact if missing
------	--------	-------------------

1 — Always provide	promo_fatigue_index, attention_context_score, break_position_pct, hour, promo_in_break	High — these drive most tree splits
2 — Provide if available	promo_title, genre_guess, break_event_position, weather_rain_mm, weather_indoor_viewing_index, ott_hook_strength	Medium — meaningful improvement in borderline
3 — Nice to have	netflix_popularity_score, synthetic_premiere_probability, weather_tmax_c, ott_dropoff_prob	Low — marginal gain at the edges

9. How to Run

9.1 Local development

```
# 1. Create and activate virtual environment
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate       # Mac / Linux

# 2. Install dependencies
pip install -r requirements.txt

# 3. (If inference_schema.json is missing) recover it
python generate_schema.py

# 4. Start the API
uvicorn main:app --reload

# 5. In a second terminal, start the UI
streamlit run streamlit_app.py

# 6. Run tests
pytest tests/
```

9.2 Docker

```
# Build and start
docker compose up --build

# Update model without rebuild
# 1. Replace pkl files in ./model/
# 2. POST http://localhost:8000/reload
```

9.3 Access points

Service	URL
API	http://localhost:8000
Swagger UI	http://localhost:8000/docs
Streamlit	http://localhost:8501

10. File Structure

```
ITV_PROMO/
■■■ ctv_promo_pipeline.py      # Training pipeline
■■■ main.py                   # FastAPI service
■■■ streamlit_app.py          # Streamlit frontend
■■■ generate_schema.py        # Schema recovery tool
■■■ generate_pdf.py           # This documentation generator
■■■ requirements.txt          # Python dependencies
■■■ Dockerfile
■■■ docker-compose.yml
■■■ .env.example
■■■ model/
■   ■■■ promo_move_classifier.pkl
■   ■■■ promo_uplift_regressor.pkl
■   ■■■ inference_schema.json
■   ■■■ eda.png
■   ■■■ feature_importance.png
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_api.py
■■■ .venv/                    # Virtual environment
```

11. Glossary

Term	Definition
should_move	Binary label: 1 = this promo would perform better if repositioned in its break.
uplift_if_optimised	Continuous label: expected effectiveness score improvement if the promo is repositioned.
PR-AUC	Area under the Precision-Recall curve. The preferred metric for imbalanced binary classification.
scale_pos_weight	XGBoost hyperparameter = $n_{\text{negative}} / n_{\text{positive}}$. Compensates for class imbalance by up-weighting the positive class.
enable_categorical	XGBoost flag that enables native handling of pandas category dtype columns, avoiding the need for one-hot encoding.
Default direction	The branch (left or right) that XGBoost takes at a tree split when the feature value is NaN. Learned from the training data.
inference_schema.json	JSON file storing the training feature list, categorical column names, and per-category vocabulary.
promo_fatigue_index	Score (0–1) measuring how 'worn out' the audience is with a particular promo. Higher = more fatigue.
attention_context_score	Score (0–1) measuring how receptive the audience in that slot is likely to be. Lower = less attention.
break_position_pct	Float 0–1 representing the promo's relative position within its break. 0=first, 1=last.
OTT	Over-the-top streaming (Netflix, Amazon, etc.). OTT signals reflect streaming competition that can impact promo performance.